

BIBLE BEAUTIFULSOUP4 – RÉFÉRENCE EXHAUSTIVE DES FONCTIONS

LÉGENDE : [bs] = BeautifulSoup | [tag] = Tag | [nav] = NavigableString | [rs] = ResultSet

[bs] Création de l'objet BeautifulSoup – parsers et instanciation

DESCRIPTION : Charger du HTML ou XML dans BeautifulSoup pour le parser.
POURQUOI : Point d'entrée unique. Le choix du parser impacte vitesse et tolérance.
CONTEXTE : Scraping, extraction de données, transformation de HTML/XML.

— PARSERS DISPONIBLES —

BeautifulSoup(html, "html.parser")	Parser Python stdlib (lent, tolérant)
BeautifulSoup(html, "lxml")	Parser lxml (rapide, tolérant) – pip install lxml
BeautifulSoup(html, "lxml-xml")	Parser XML strict via lxml
BeautifulSoup(html, "xml")	Alias de lxml-xml
BeautifulSoup(html, "html5lib")	Parser html5lib (très tolérant, lent) – pip install html5lib

— SOURCES D'ENTRÉE —

BeautifulSoup(" texte ", "html.parser")	Depuis une chaîne HTML
BeautifulSoup(b" texte ", "html.parser")	Depuis des bytes
BeautifulSoup(open("page.html"), "html.parser")	Depuis un fichier ouvert
BeautifulSoup(response.text, "html.parser")	Depuis requests
BeautifulSoup(response.content, "html.parser")	Depuis bytes requests

— OPTIONS AVANCÉES —

BeautifulSoup(html, "lxml", from_encoding="latin-1")	Encodage source explicite
BeautifulSoup(html, "html.parser", exclude_encodings=["utf-8"])	Exclure des encodages
BeautifulSoup(html, "lxml", multi_valued_attributes=None)	Désactive les attributs multi-valeurs
BeautifulSoup(html, "lxml", element_classes={Tag: MaClasse})	Classes personnalisées

COMPARATIF PARSERS :

html.parser	Stdlib, pas de dépendance, légèrement différent de lxml
lxml	Le plus rapide, très recommandé, traite HTML mal formé
html5lib	Identique à un navigateur, très lent, meilleure tolérance
lxml-xml	XML strict, sensible à la casse, respecte les namespaces

EXEMPLES :

```
from bs4 import BeautifulSoup
import requests

r = requests.get("https://example.com")
soup = BeautifulSoup(r.text, "lxml")

with open("page.html", encoding="utf-8") as f:
    soup = BeautifulSoup(f, "html.parser")

soup = BeautifulSoup("<html><body><p>test</p></body></html>", "lxml")
```

[bs/tag] Recherche d'éléments – find() et find_all()

DESCRIPTION : Trouver des balises dans l'arbre HTML.
POURQUOI : Méthodes centrales de bs4 pour extraire des éléments ciblés.
CONTEXTE : Extraction de titres, liens, tableaux, formulaires, etc.

— FIND – PREMIER RÉSULTAT —

soup.find("p")	Premier <p> trouvé (ou None)
soup.find("a")	Premier <a>
soup.find("div", class_="menu")	Premier <div class="menu">
soup.find("div", {"class": "menu"})	Idem via dict d'attributs
soup.find("a", href=True)	Premier <a> ayant un attribut href
soup.find("input", {"type": "text"})	Premier <input type="text">
soup.find("p", id="intro")	Premier <p id="intro">
soup.find(id="intro")	Cherche n'importe quelle balise avec cet id
soup.find(True)	Première balise quelconque
soup.find(string="texte exact")	Noeud texte exact
soup.find(string=re.compile("motif"))	Noeud texte par regex
soup.find(["h1", "h2", "h3"])	Premier h1, h2 ou h3 trouvé

— FIND_ALL – TOUS LES RÉSULTATS	
<code>soup.find_all("p")</code>	Tous les <p> → liste
<code>soup.find_all(["h1", "h2"])</code>	Tous les h1 et h2
<code>soup.find_all("a", href=True)</code>	Tous les <a> avec href
<code>soup.find_all("a", href=re.compile("^http"))</code>	Liens commençant par http
<code>soup.find_all("div", class_="item")</code>	Tous les <div class="item">
<code>soup.find_all(True)</code>	Toutes les balises
<code>soup.find_all(string=re.compile("prix"))</code>	Tous les textes contenant "prix"
<code>soup.find_all("p", limit=5)</code>	Maximum 5 résultats
<code>soup.find_all("p", recursive=False)</code>	Enfants directs seulement
<code>soup.find_all(attrs={"data-id": True})</code>	Présence d'un attribut data
<code>soup.find_all(attrs={"data-type": "card"})</code>	Attribut data avec valeur précise
— RECHERCHE PAR FONCTION	
<code>soup.find_all(lambda tag: tag.name == "p" and tag.get("class"))</code>	Filtre par fonction personnalisée
<code>soup.find_all(lambda tag: len(tag.attrs) == 2)</code>	Balises avec exactement 2 attributs
— ALIAS ET RACCOURCIS	
<code>soup("p")</code>	Alias de <code>soup.find_all("p")</code>
<code>soup("a", limit=10)</code>	Alias avec options
<code>tag.find_all("span")</code>	Cherche dans un sous-arbre
<code>tag.find("b")</code>	find() dans un sous-arbre

```

EXEMPLES      :
# Tous les liens d'une page
liens = [a["href"] for a in soup.find_all("a", href=True)]

# Premier paragraphe avec classe "intro"
intro = soup.find("p", class_="intro")

# Titre principal
h1 = soup.find("h1")
texte = h1.get_text() if h1 else ""

# Balises avec attribut data-*
cartes = soup.find_all(attrs={"data-type": "card"})

```

[bs/tag] Sélecteurs CSS – select() et select_one()

DESCRIPTION : Utiliser les sélecteurs CSS pour cibler des éléments.
POURQUOI : Syntaxe familière aux développeurs web, très expressive.
CONTEXTE : Ciblage précis, pseudos-classes, sélecteurs combinés.

— SELECT – TOUS LES RÉSULTATS	
<code>soup.select("p")</code>	Tous les <p>
<code>soup.select(".classe")</code>	Tous les éléments de classe "classe"
<code>soup.select("#mon-id")</code>	Élément avec id="mon-id"
<code>soup.select("div.menu")</code>	<div class="menu">
<code>soup.select("div > p")</code>	<p> enfant direct de <div>
<code>soup.select("div p")</code>	<p> descendant de <div>
<code>soup.select("a[href]")</code>	<a> ayant un href
<code>soup.select("a[href^='http']")</code>	href commençant par "http"
<code>soup.select("a[href\$='.pdf']")</code>	href finissant par ".pdf"
<code>soup.select("a[href*='example']")</code>	href contenant "example"
<code>soup.select("input[type='text']")</code>	<input type="text">
<code>soup.select("h1, h2, h3")</code>	h1 OU h2 OU h3
<code>soup.select("ul li")</code>	dans
<code>soup.select("ul > li")</code>	enfant direct de
<code>soup.select("li:first-child")</code>	Premier
<code>soup.select("li:last-child")</code>	Dernier
<code>soup.select("li:nth-child(2)")</code>	Deuxième
<code>soup.select("li:nth-of-type(odd)")</code>	impairs
<code>soup.select("p:not(.intro)")</code>	<p> sans classe "intro"
<code>soup.select("a + span")</code>	 immédiatement après <a>
<code>soup.select("a ~ span")</code>	 après <a> (sibling)
<code>soup.select("[data-id]")</code>	Tout élément avec data-id
<code>soup.select("[class~='active']")</code>	Classe parmi plusieurs
— SELECT_ONE – PREMIER RÉSULTAT	
<code>soup.select_one("h1")</code>	Premier h1 (ou None)

soup.select_one(".prix")	Premier élément .prix
soup.select_one("#header nav a")	Premier lien dans nav de #header
tag.select_one("span.badge")	Dans un sous-arbre

```

EXEMPLES      :
# Tous les liens de navigation
nav_links = soup.select("nav ul li a")

# Prix dans une fiche produit
prix = soup.select_one(".product .price")
valeur = prix.get_text(strip=True) if prix else "N/A"

# Tableau de données
lignes = soup.select("table tbody tr")
for ligne in lignes:
    cellules = [td.get_text(strip=True) for td in ligne.select("td")]

```

[tag] Accès aux attributs d'une balise – attrs / get / []

DESCRIPTION : Lire et modifier les attributs HTML d'une balise.
POURQUOI : Extraire href, src, class, id, data-*, etc.
CONTEXTE : Scraping de liens, images, attributs personnalisés.

LECTURE :	
tag["href"]	Valeur de l'attribut (KeyError si absent)
tag.get("href")	Valeur ou None si absent
tag.get("href", "")	Valeur ou valeur par défaut
tag.attrs	Dict de tous les attributs
tag.attrs.get("class", [])	Classe(s) sous forme de liste
tag["class"]	Liste des classes (ex: ["menu", "active"])
tag.get("id")	Valeur de l'id
tag.has_attr("href")	True si l'attribut existe
tag.name	Nom de la balise ("div", "a"...)

MODIFICATION :	
tag["href"] = "https://nouveau.com"	Modifie l'attribut
tag["class"] = ["nouvelle-classe"]	Remplace les classes
del tag["class"]	Supprime l'attribut

ATTRIBUTS MULTI-VALEURS :	
tag["class"]	Retourne une liste ["a", "b"]
" ".join(tag.get("class", []))	Recomposer la string de classes

```

EXEMPLES      :
for a in soup.find_all("a", href=True):
    print(a["href"], a.get_text(strip=True))

for img in soup.find_all("img"):
    src = img.get("src", "")
    alt = img.get("alt", "")
    print(src, alt)

# Vérifier une classe
div = soup.find("div")
if div and "active" in div.get("class", []):
    print("Div active")

```

[tag] Extraction du texte – get_text() / strings / string

DESCRIPTION : Extraire le contenu textuel d'une balise ou de l'arbre entier.
POURQUOI : Récupérer le texte visible, nettoyer le HTML.
CONTEXTE : NLP, extraction de contenu, nettoyage de données.

FONCTIONS PRINCIPALES :	
tag.get_text()	Tout le texte (avec espaces/sauts)
tag.get_text(strip=True)	Texte nettoyé des espaces
tag.get_text(separator=" ")	Texte avec séparateur entre noeuds
tag.get_text(separator="\n", strip=True)	Texte ligne par ligne nettoyé
tag.string	Texte si un seul noeud texte (sinon None)
tag.strings	Générateur de tous les textes
tag.striped_strings	Générateur de textes sans espaces vides

<code>list(tag.strings)</code>	Liste de tous les textes
<code>list(tag.striped_strings)</code>	Liste nettoyée
<code>tag.text</code>	Alias de <code>get_text()</code> (sans options)
<code>nav_string.string</code>	Contenu d'un NavigableString

NAVIGABLESTRING :	
<code>from bs4 import NavigableString</code>	
<code>isinstance(tag.string, NavigableString)</code>	Vérifier si c'est un noeud texte
<code>str(tag.string)</code>	Convertir en str Python

EXEMPLES :

```
# Texte propre d'un article
article = soup.find("article")
texte = article.get_text(separator="\n", strip=True)

# Extraire tous les mots du document
mots = soup.get_text().split()

# Concaténer avec séparateur
paragraphes = [p.get_text(strip=True) for p in soup.find_all("p")]
```

[tag] Navigation dans l'arbre – parents / children / siblings

DESCRIPTION : Se déplacer dans l'arbre HTML : parents, enfants, voisins.
POURQUOI : Naviguer relativement à un élément trouvé.
CONTEXTE : Extraction de données structurées, tableaux, listes imbriquées.

— ENFANTS —	
<code>tag.children</code>	Générateur des enfants directs
<code>list(tag.children)</code>	Liste des enfants (Tag + NavigableString)
<code>tag.contents</code>	Liste des enfants directs (identique)
<code>tag.contents[0]</code>	Premier enfant
<code>tag.contents[-1]</code>	Dernier enfant
<code>tag.descendants</code>	Générateur de TOUS les descendants
<code>tag.find("span")</code>	Premier span parmi les descendants

— PARENTS —	
<code>tag.parent</code>	Parent direct
<code>tag.parents</code>	Générateur de tous les ancêtres
<code>list(tag.parents)</code>	Liste des ancêtres
<code>tag.parent.parent</code>	Grand-parent

— SIBLINGS (VOISINS) —	
<code>tag.next_sibling</code>	Prochain voisin (peut être texte)
<code>tag.previous_sibling</code>	Voisin précédent (peut être texte)
<code>tag.next_siblings</code>	Générateur des voisins suivants
<code>tag.previous_siblings</code>	Générateur des voisins précédents
<code>list(tag.next_siblings)</code>	Liste des voisins suivants

— ÉLÉMENTS SUIVANTS / PRÉCÉDENTS —	
<code>tag.next_element</code>	Noeud suivant dans l'arbre (DFS)
<code>tag.previous_element</code>	Noeud précédent dans l'arbre
<code>tag.next_elements</code>	Générateur des noeuds suivants
<code>tag.previous_elements</code>	Générateur des noeuds précédents

— RACCOURCIS DE NAVIGATION —	
<code>soup.html</code>	Accès direct à <html>
<code>soup.head</code>	Accès direct à <head>
<code>soup.body</code>	Accès direct à <body>
<code>soup.title</code>	Accès direct à <title>
<code>soup.title.string</code>	Texte du titre
<code>tag.p</code>	Premier <p> enfant direct
<code>tag.a</code>	Premier <a> enfant direct

EXEMPLES :

```
# Siblings balise (ignorer les noeuds texte)
from bs4 import Tag
for sibling in tag.next_siblings:
    if isinstance(sibling, Tag):
        print(sibling)

# Remonter jusqu'au parent d'une classe
el = soup.find("span", class_="prix")
```

```
fiche = el.find_parent("div", class_="produit")

# Tous les enfants Tag (sans les NavigableString)
enfants = [c for c in tag.children if isinstance(c, Tag)]
```

[tag] Recherche par navigation – find_parent / find_next / find_previous
--

DESCRIPTION : Variantes de find() qui naviguent dans des directions spécifiques.
 POURQUOI : Trouver des éléments relatifs à un noeud donné.

FONCTIONS PRINCIPALES :

tag.find_parent("div")	Premier ancêtre <div>
tag.find_parent("div", class_="wrap")	Ancêtre avec classe
tag.find_parents("li")	Tous les ancêtres
tag.find_next_sibling("p")	Prochain voisin <p>
tag.find_next_siblings("p")	Tous les voisins <p> suivants
tag.find_previous_sibling("p")	Voisin <p> précédent
tag.find_previous_siblings("p")	Tous les voisins <p> précédents
tag.find_next("p")	Prochain <p> dans l'arbre (DFS)
tag.find_all_next("p")	Tous les <p> suivants
tag.find_previous("p")	<p> précédent dans l'arbre
tag.find_all_previous("p")	Tous les <p> précédents

EXEMPLES :

```
# Trouver la ligne de tableau contenant un élément
cellule = soup.find("td", string="Total")
ligne = cellule.find_parent("tr")
valeurs = [td.get_text(strip=True) for td in ligne.find_all("td")]

# Récupérer le libellé d'un champ (avant un input)
champ = soup.find("input", {"name": "email"})
label = champ.find_previous_sibling("label")
```

[tag] Modification de l'arbre – insert / append / decompose / replace

DESCRIPTION : Créer, insérer, déplacer, modifier ou supprimer des éléments.
 POURQUOI : Transformer du HTML, nettoyer, enrichir.
 CONTEXTE : Post-traitement HTML, sanitisation, génération de contenu.

— CRÉATION —

```
from bs4 import BeautifulSoup, Tag, NavigableString
nouvelle_balise = soup.new_tag("a", href="https://example.com")
nouveau_texte = soup.new_string("Mon texte")
nouveau_comment = soup.new_string("commentaire", Comment)
```

— INSERTION —

tag.append(enfant)	Ajoute à la fin des enfants
tag.insert(0, enfant)	Insère en position 0
tag.insert_before(element)	Insère avant la balise courante
tag.insert_after(element)	Insère après la balise courante
tag.prepend(enfant)	Insère au début (Python 3 / bs4 4.9+)

— MODIFICATION —

tag.string = "Nouveau texte"	Remplace le texte
tag.string.replace_with("Nouveau texte")	Remplace un NavigableString
tag.replace_with(autre_tag)	Remplace la balise par une autre
tag.smooth()	Fusionne les NavigableStrings adjacents
tag.encode(formatter="minimal")	Sérialise en bytes
tag.decode(formatter="minimal")	Sérialise en str

— SUPPRESSION —

tag.decompose()	Supprime et détruit le tag de l'arbre
tag.extract()	Supprime et retourne le tag
tag.clear()	Vide le contenu du tag (garde le tag)
tag.unwrap()	Supprime le tag, garde les enfants

EXEMPLES :

```
# Ajouter un lien dans un div
div = soup.find("div", id="liens")
lien = soup.new_tag("a", href="https://example.com")
lien.string = "Exemple"
```

```
div.append(lien)

# Supprimer tous les scripts
for script in soup.find_all("script"):
    script.decompose()

# Remplacer un tag par son contenu (unwrap)
for span in soup.find_all("span", class_="inutile"):
    span.unwrap()
```

[bs/tag] Sérialisation – prettify / str / encode |

DESCRIPTION : Convertir l'arbre bs4 en chaîne HTML ou bytes.
 POURQUOI : Sauvegarder, afficher, transmettre le HTML modifié.

FONCTIONS PRINCIPALES :

str(soup)	HTML complet en str
soup.prettify()	HTML indenté lisible (str)
soup.prettify(encoding="utf-8")	HTML indenté en bytes
soup.encode()	HTML en bytes (utf-8 par défaut)
soup.encode("latin-1")	HTML en bytes latin-1
soup.decode()	HTML en str
str(tag)	HTML du tag et descendants
tag.prettify()	HTML indenté du tag
tag.encode_contents()	Contenu intérieur en bytes
tag.decode_contents()	Contenu intérieur en str

FORMATTERS :

soup.prettify(formatter="minimal")	Échappe < > & seulement
soup.prettify(formatter="html")	Échappe tous les caractères spéciaux HTML
soup.prettify(formatter="html5")	Formatage HTML5
soup.prettify(formatter=None)	Aucun échappement
soup.prettify(formatter="xml")	Formatage XML

EXEMPLES :

```
# Sauvegarder le HTML modifié
with open("sortie.html", "w", encoding="utf-8") as f:
    f.write(str(soup))

# Afficher un extrait lisible
print(soup.find("table").prettify())
```

[bs] Types spéciaux de noeuds – Comment / CDATA / ProcessingInstruction |

DESCRIPTION : Noeuds HTML/XML autres que les balises et textes simples.
 POURQUOI : Traiter les commentaires HTML, les sections CDATA, les instructions XML.

TYPES :

```
from bs4 import Comment, CDATA, ProcessingInstruction, Declaration, Doctype
```

Comment	<!-- commentaire HTML -->
CDATA	<![CDATA[...]]>
ProcessingInstruction	<?xml version="1.0"?>
Declaration	<!DOCTYPE html>
Doctype	Sous-type de Declaration
NavigableString	Noeud texte ordinaire
Tag	Balise HTML/XML

DÉTECTION :

isinstance(noeud, Comment)	Teste si commentaire
isinstance(noeud, NavigableString)	Teste si texte
isinstance(noeud, Tag)	Teste si balise
type(noeud).__name__	Nom du type

EXEMPLES :

```
from bs4 import Comment

# Trouver et supprimer tous les commentaires
for commentaire in soup.find_all(string=lambda t: isinstance(t, Comment)):
    commentaire.extract()
```

```
# Lire un commentaire conditionnel IE
commentaires = soup.find_all(string=isinstance(Comment))
```

[bs] Recherche par regex – re.compile avec find_all

DESCRIPTION : Utiliser des expressions régulières dans les recherches bs4.
 POURQUOI : Correspondances partielles sur noms de balises, attributs, textes.
 CONTEXTE : Attributs dynamiques, classes multiples, textes variables.

FONCTIONS PRINCIPALES :

```
import re
soup.find_all(re.compile("^h[1-6]$"))      Tous les titres h1-h6
soup.find_all("a", href=re.compile("^http")) Liens externes
soup.find_all("a", href=re.compile(r"\.pdf$")) Liens vers PDF
soup.find_all(string=re.compile("prix"))    Textes contenant "prix"
soup.find_all("div",
              class_=re.compile("col-\d+")) Classes col-1, col-2, col-12...
soup.find(string=re.compile(r"\d+€"))       Texte avec prix
```

EXEMPLES :

```
# Tous les titres de la page
titres = soup.find_all(re.compile("^h[1-6]$"))
for t in titres:
    print(t.name, t.get_text(strip=True))

# Liens vers des domaines spécifiques
liens_wiki = soup.find_all("a", href=re.compile(r"wikipedia\.org"))
```

[bs] Extraction de tableaux – table / tr / td / th

DESCRIPTION : Extraire des données structurées depuis des tableaux HTML.
 POURQUOI : Cas très fréquent en scraping (données financières, comparatifs...).
 CONTEXTE : Tables HTML simples et imbriquées.

PATTERN COURANT :

```
table = soup.find("table")
lignes = table.find_all("tr")

# En-têtes
entetes = [th.get_text(strip=True) for th in lignes[0].find_all("th")]

# Données
donnees = []
for ligne in lignes[1:]:
    cellules = [td.get_text(strip=True) for td in ligne.find_all("td")]
    if cellules:
        donnees.append(dict(zip(entetes, cellules)))
```

AVEC PANDAS :

```
import pandas as pd
# Méthode directe (plus rapide)
dfs = pd.read_html(str(soup))      Extrait toutes les tables
dfs = pd.read_html(str(table))     Extrait une table spécifique
df = dfs[0]                        Première table
```

EXEMPLES :

```
# Table avec rowspan/colspan (plus complexe)
table = soup.find("table", id="resultats")
for tr in table.select("tbody tr"):
    cols = tr.find_all(["td", "th"])
    vals = [c.get_text(strip=True) for c in cols]
    print(vals)
```

[bs] Cas d'usage fréquents – patterns de scraping

DESCRIPTION : Recettes prêtes à l'emploi pour les tâches de scraping courantes.

— LIENS —

```
# Tous les liens absolus
```

```

liens = [a["href"] for a in soup.find_all("a", href=re.compile("^http"))]

# Liens internes
liens_internes = [a["href"] for a in soup.find_all("a", href=True)
                  if not a["href"].startswith("http")]

# Liens + texte
[(a.get_text(strip=True), a["href"]) for a in soup.find_all("a", href=True)]

— IMAGES —
# Toutes les images avec src et alt
images = [(img.get("src", ""), img.get("alt", ""))
          for img in soup.find_all("img")]

# Images avec attribut data-src (lazy loading)
images = soup.find_all("img", attrs={"data-src": True})
srcs = [img["data-src"] for img in images]

— FORMULAIRES —
form = soup.find("form")
action = form.get("action", "")
method = form.get("method", "get")
champs = {
    inp["name"]: inp.get("value", "")
    for inp in form.find_all("input")
    if inp.get("name")
}

— MÉTADONNÉES —
titre = soup.title.string if soup.title else ""
desc = soup.find("meta", attrs={"name": "description"})
contenu = desc["content"] if desc else ""

og_title = soup.find("meta", property="og:title")
og_image = soup.find("meta", property="og:image")

# Tous les meta
metas = {m.get("name", m.get("property", "")): m.get("content", "")
        for m in soup.find_all("meta")}

— NETTOYAGE HTML —
# Supprimer scripts, styles, commentaires
for tag in soup(["script", "style", "noscript"]):
    tag.decompose()
for c in soup.find_all(string=lambda t: isinstance(t, Comment)):
    c.extract()

# Texte propre de la page
texte = soup.get_text(separator="\n", strip=True)
lignes = [l for l in texte.splitlines() if l.strip()]

— JSON-LD / DONNÉES STRUCTURÉES —
import json
scripts = soup.find_all("script", type="application/ld+json")
for s in scripts:
    data = json.loads(s.string)
    print(data)

```

===== RÉCAPITULATIF BS4 – MÉTHODES ET ATTRIBUTS CLÉS =====

CATÉGORIE	MÉTHODE / ATTRIBUT	USAGE
Instanciation	BeautifulSoup(html, parser)	Créer l'objet soup
Recherche 1er	.find(tag, attrs)	Premier résultat ou None
Recherche tous	.find_all(tag, attrs, limit)	Liste de résultats
Sélecteur CSS 1er	.select_one(css)	Premier résultat CSS
Sélecteurs CSS	.select(css)	Liste résultats CSS
Attribut	tag["attr"] / tag.get("attr")	Lire un attribut HTML
Tous attributs	tag.attrs	Dict des attributs
Texte	.get_text(strip, separator)	Extraire le texte
Texte unique	.string	Texte si un seul noeud
Textes multiples	.stripped_strings	Générateur textes nettoyés

Enfants directs	.children / .contents	Générateur / liste enfants
Tous descendants	.descendants	Générateur descendants
Parent	.parent / .parents	Remonter l'arbre
Voisins	.next_sibling / .previous_sibling	Naviguer latéralement
Ancêtre ciblé	.find_parent(tag)	Premier ancêtre du type
Prochain	.find_next(tag)	Prochain tag dans l'arbre
Nom de la balise	.name	"div", "a", "p"...
Sérialisation	str(tag) / tag.prettify()	HTML en string
Ajout enfant	.append() / .insert()	Modifier l'arbre
Suppression	.decompose() / .extract()	Retirer un noeud
Vider	.clear()	Vider les enfants
Déballer	.unwrap()	Garder le contenu, enlever le tag
Nouveau tag	soup.new_tag(name, **attrs)	Créer une nouvelle balise
Regex	re.compile("motif")	Filtre regex sur find_all

— COMPARATIF find() vs select() —

BESOIN

UTILISER

Balise simple, attribut simple	.find() / .find_all()
Sélecteur CSS complexe	.select() / .select_one()
Filtre par fonction lambda	.find_all(fonction)
Filtre par regex	.find_all(re.compile(...))
Navigation relative à un noeud	.find_parent() / .find_next()
Pseudo-classes CSS (:nth-child, :not...)	.select()

=====